

PACKAGE CLAUSE

You can use *package clause* in a *file* to let Go know that which *package* that *file* belongs to.

package clause ←

this should be the first code

it can only appear once

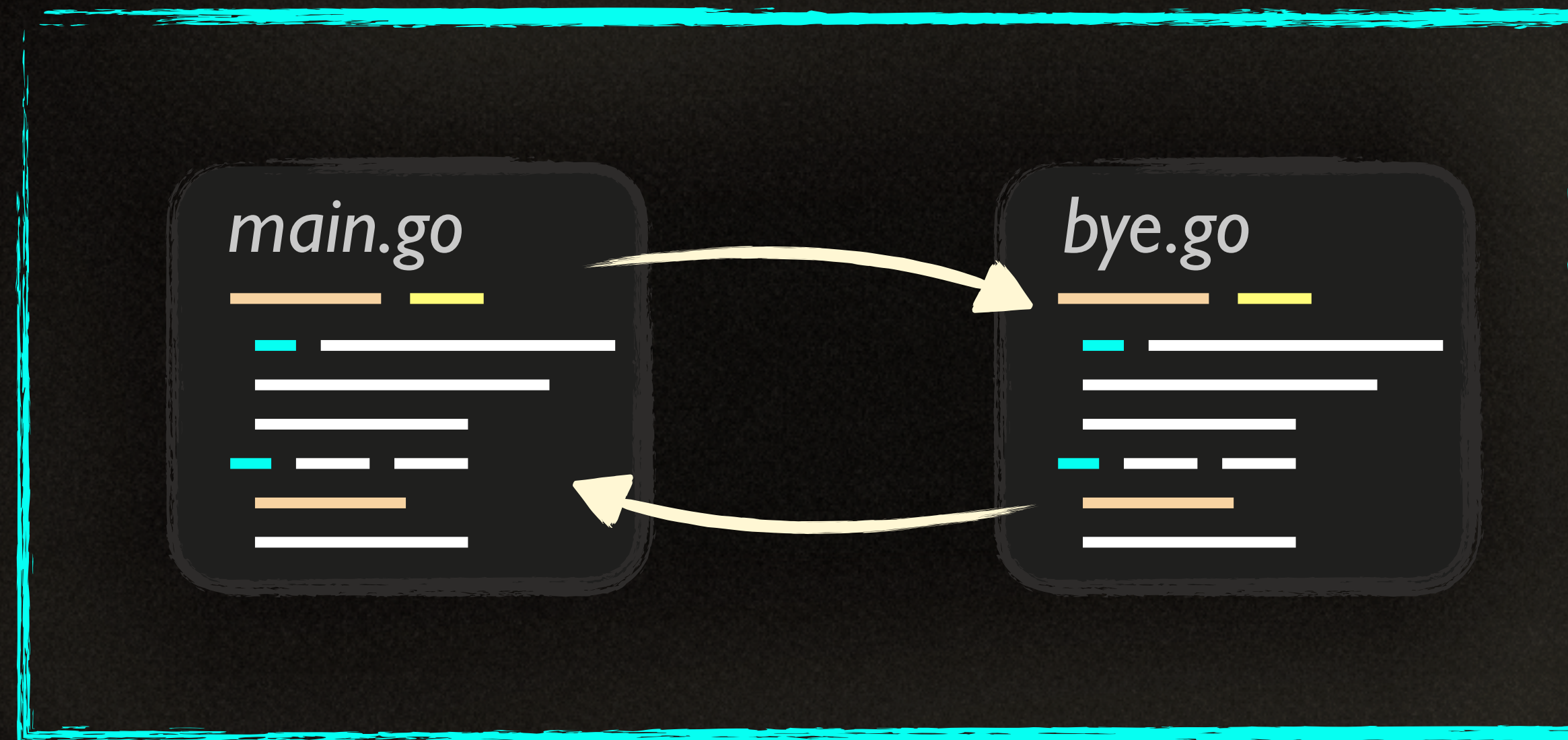
```
package main
import "fmt"

func main() {
    fmt.Println("Hello!")
}
```


PACKAGE SCOPE

Declarations which are outside of functions are visible to the files belong to the same package.

main package



IMPORTING

Importing is like as if you've declared what's inside the imported package's files in your own file.

myfile.go

```
import "fmt"
import "errors"
import "time"
```

format on
print.go

package fmt

errors.go

package errors

bye.go

hey.go

package
your

time.go

package time

PACKAGE SCOPE

Each Go package has its own *scope*. For example, declared *funcs* are only *visible* to the files belong to the same package.

"**main()**" is visible
throughout
the main package ←

```
package main
import "fmt"

func main() {
    fmt.Println("Hello!")
}
```


SCOPE

Every line of code can have *different scope* depending on their *position* in a Go file.

file scoped ←
only visible in this file

package scoped ←
visible to all the files
belong to the package

other packages can't
see them

```
package main
```

```
import "fmt"
```

```
const ok = true
```

```
func main() {
```

```
    var hello = "Hello!"  
    fmt.Println(hello, ok)
```

```
}
```

→ **block** scoped
declaration

only visible
after its declaration
until "}"

SCOPE

Every line of code can have *different scope* depending on their *position* in a Go file.

```
package main
import "fmt"
```

```
func nope() {
    const ok = true
    var hello = "Hello!"
    _ = hello
}
```

→ **block** scoped
declaration

*only visible
in "nope" func*

```
func main() {
    fmt.Println(hello, ok)
}
```


PACKAGE SCOPE

Each Go package has its own *scope*

For example, declared *funcs* are *visible* in the same package

main.go

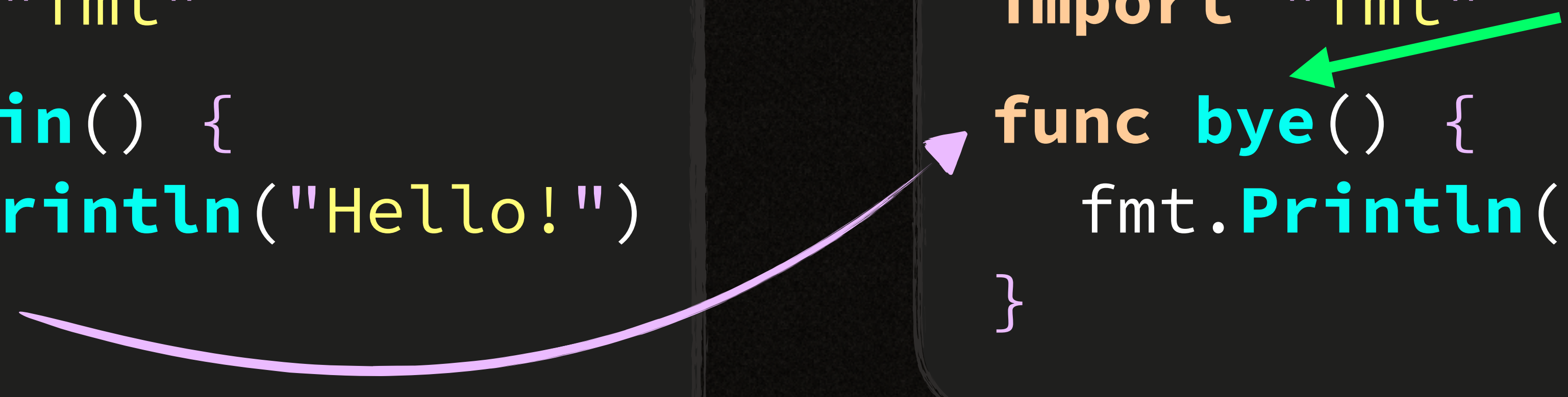
```
package main
import "fmt"

func main() {
    fmt.Println("Hello!")
    bye()
}
```

bye.go

```
package main
import "fmt"

func bye() {
    fmt.Println("Bye!")
}
```



FILE SCOPE

Each Go file has its own *scope*

Imported packages are only *visible* to the importing *file*

main.go

"**fmt**" is visible
throughout
the file

```
package main
import "fmt"

func main() {
    fmt.Println("Hello!")
}
```


FILE SCOPE

Each Go file has its own *scope*

Imported *packages* are only *visible* to the importing *file*

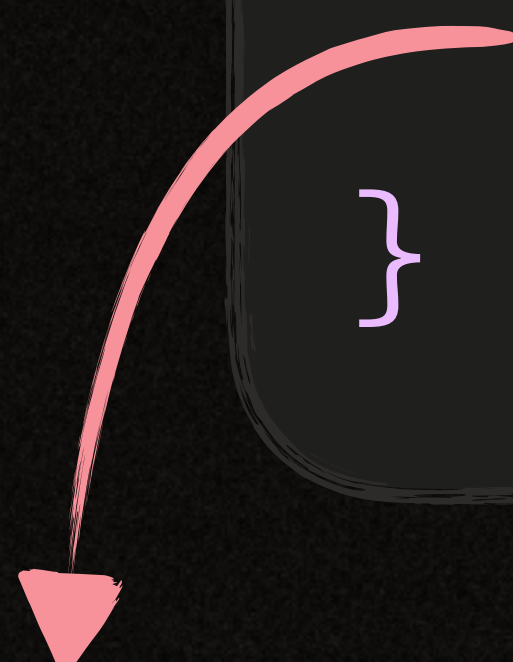
bye.go

```
package main
```

```
func bye() {
```

```
    fmt.Println("Bye!")
```

```
}
```



bye.go can't use a package that it didn't import

FILE SCOPE

*Each Go file has its own **scope***

*Imported **packages** are only **visible** to the importing **file***



```
package main
import "fmt"

func bye() {
    fmt.Println("Bye!")
}
```


FILE SCOPE

Each file has to import external packages on its own

main package



learn-go/first/explain/importing

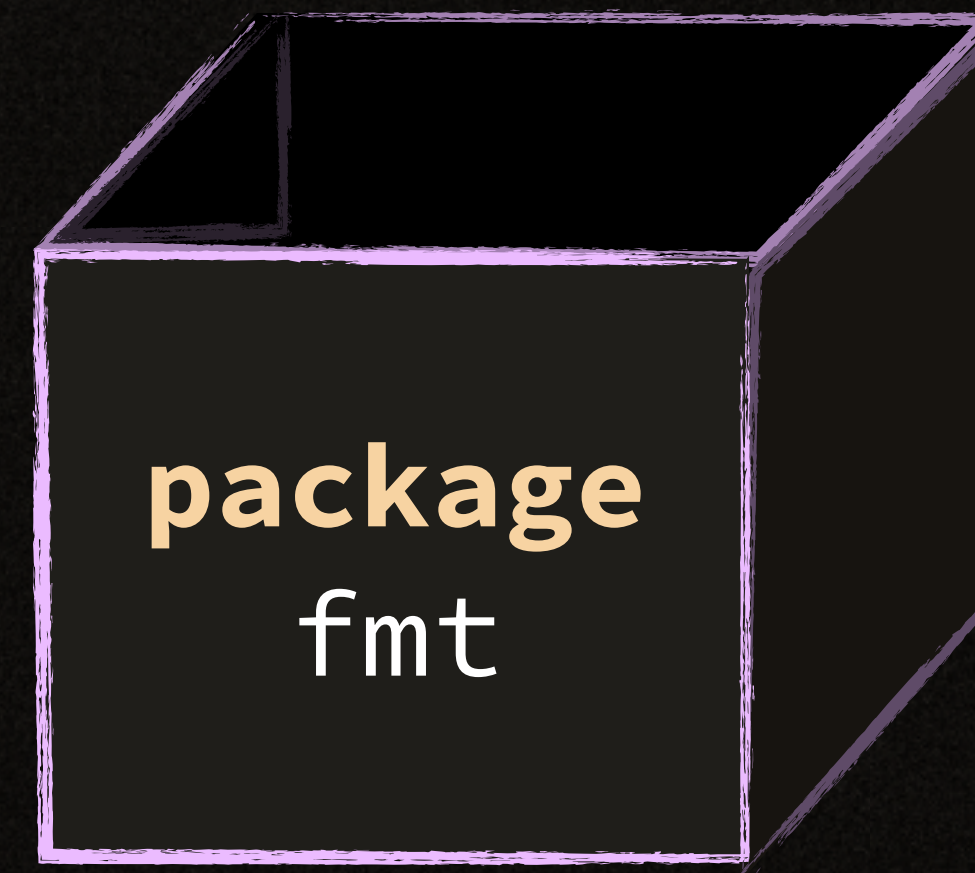
PACKAGE KINDS

There are two kinds of packages in Go: *Executable* and *Library*.

**Executable
Package**



**Library
Package**



+

func main()

LIBRARY

created for **reusability**

non-executable

importable

can have **any name**

no **func main**

EXECUTABLE

created for **running**

executable

non-importable

name should be **main**

func main